Multi-Bottleneck Fairness for High-Precision Congestion Control

A Minimal Switch Extension for Multi-Bottleneck Max-Min Fairness

Haoyu Wang
University of Massachusetts Boston
Boston, MA, USA
haoyu.wang001@umb.edu

Bo Sheng
University of Massachusetts Boston
Boston, MA, USA
bo.sheng@umb.edu

Zerin Shaima Meem
University of Nebraska Omaha
Omaha, NE, USA
zmeem@unomaha.edu

Xiaoqian Zhang
University of Nebraska Omaha
Omaha, NE, USA
xiaoqianzhang@unomaha.edu

Abstract

HPCC (SIGCOMM '19) achieves near-optimal flow completion time for single-bottleneck RDMA workloads via INT-based precision congestion control with no per-flow switch state. We show its design has a structural fairness failure on *multi-bottleneck* paths — increasingly common in cross-pod and disaggregated datacenter traffic: a flow crossing several shared bottlenecks is starved by ∼2× relative to single-bottleneck competitors, and fails entirely once the per-hop INT record overflows its hop budget. Through a five-variant sender-side ablation we show the gap is not closable from the endpoint — a sender cannot compute its max-min fair share because the link's flow count is not observable in INT. The fix belongs at the switch. HPCC-FS adds a single RCP-style fair-rate signal — one 64-bit header field and one scalar of state per egress port — whose fixed point is the max-min share, reached *without ever counting flows*. It drives the multi-bottleneck penalty to ≈1.0× across bottleneck counts, asymmetric flow sizes, staggered starts, and ECMP fat-trees, with zero PFC. We are explicit that the mechanism is RCP; our primary contribution is the *diagnosis* and the demonstration that the fix belongs in the network, not the endpoint. The stock HPCC code path is left byte-for-byte unchanged.

1 Introduction

HPCC [15] and its derivatives represent the state of the art in datacenter RDMA congestion control. Their design philosophy is unusually clean: the switch stamps raw per-hop state (transmitted bytes, queue length, link rate) into INT headers; the sender computes a precise utilization estimate and applies a multiplicative-increase, multiplicative-decrease update toward a target utilization η. The switch is stateless per flow; the wire format adds only a few bytes per hop; and on the canonical single-bottleneck benchmarks HPCC matches or beats DCQCN, TIMELY, and DCTCP on flow completion time and queueing.

Modern datacenter workloads, however, increasingly involve traffic that traverses *multiple contended links* in series: cross-pod ML allreduce, disaggregated memory and storage, and multi-tenant flows that share aggregation and core layers. Whether HPCC's design still delivers fair, low-latency service in these settings has not been carefully measured.

We address that question with a deterministic, oracle-grounded methodology. Our contributions:

(1) **An empirical characterization** of HPCC's multi-bottleneck behaviour using a parameterized parking-lot benchmark and a fluid max-min oracle (§2). We show the multi-bottleneck penalty is real (∼1.9× at $N = 2$–4), worse for small flows (3.5×), robust to flow-size asymmetry and start staggering, and *RTT-independent*: equalizing the long flow's RTT with the cross flows' does not move the penalty.

(2) **A negative result for sender-only fixes** (§3). We implement five sender-side variants of HPCC ranging from a k-aware additive boost to a global share-aware update rule, and show all five fail to close the gap. We give a structural explanation: at the multi-bottleneck equilibrium every flow holds its current rate (because the bottleneck utilization sits at η, making the multiplicative term ≈1), and a sender cannot compute the strong multiplicative correction needed without knowing the link's flow count N.

(3) HPCC-FS, **a switch-minimal design that instantiates the fix with RCP-style control** (§4). On contended flows HPCC-FS does *not* use HPCC's per-hop, window-bounded control law; instead each switch port runs a standard RCP [9] fair-rate update on aggregate load and queue, advertises that single rate in the INT header, and the sender adopts the path-minimum. The switch addition is minimal — one uint64 field in a new INT header mode and one scalar of state per egress port — which is where prior INT-based designs spend their complexity; the full deployment delta is slightly larger (a new sender ACK handler, and, in our prototype, rate-only operation with the per-flow window cap disabled). We are explicit that the mechanism is RCP, not a new control law: our contribution is showing that placing this signal at the switch is what closes the gap, and that doing so is cheap and leaves the existing HPCC code path (`cc_mode 3`) byte-for-byte unchanged.

(4) **An evaluation showing HPCC-FS achieves the goal** (§5): max-min penalty ≈1.005× at $N = 4$, zero PFC events, robust across asymmetric workloads, generalizing to a tree-fabric and to a $k = 4$ ECMP fat-tree.

Our framing positions the negative result (sender-only designs are insufficient in our experiments) as a *motivation* for the positive contribution (a minimal switch extension that suffices on our benchmarks). To our knowledge, this is the first quantitative evidence that fair-share computation is hard to place purely at the sender in

INT-style precision congestion control; we do not claim a formal impossibility (see Section 3).

Artifacts. The implementation, the topology/workload generators, the max-min oracle, and deterministic scripts that regenerate the figures (`make_figures.py`) and the ablation/sensitivity/baseline tables (`run_sensitivity.py`) from source are available in our artifact-review repository.[1]

2 Background and Motivation

2.1 HPCC, briefly

HPCC's per-hop utilization estimate for hop i is

$$u_i = \frac{txRate_i}{C_i} + \frac{\min(qlen_i, qlen_i^{\text{prev}}) \cdot maxRate}{C_i \cdot W} \tag{1}$$

where the first term reflects bandwidth utilization and the second is a queue-drain term that keeps standing queues bounded. In single-rate mode the sender takes $U = \max_i u_i$, smooths it over the base RTT, and applies

$$new\_rate = \begin{cases} curRate/\max c + R_{AI} & \max c \geq 1 \\ curRate + R_{AI} & \text{otherwise} \end{cases}$$

with $\max c = U/\eta$. The window W is sized to $m_{win} \cdot rate/NIC\_line$ under `var_win`.

2.2 The multi-bottleneck parking lot

The *parking lot* is a classic congestion-control fairness benchmark [5, 12]. The name is an analogy: a long boulevard (the long flow's path) is crossed by a series of side streets, and at each intersection a different car (a single-link cross flow) merges on and exits. Because every link is equally contended, the *max-min fair* allocation [5] gives every flow an equal share, so the long flow should run exactly as fast as each cross flow despite traversing many contention points rather than one. The parking lot therefore isolates the multi-bottleneck fairness question in its simplest form: any deviation of the long flow's rate from the cross flows' is pure multi-bottleneck unfairness, with no confounding asymmetry in link capacity or hop count of the competitors.

We construct a parameterized parking-lot benchmark with N inter-switch bottleneck links (25 Gbps each) and 100 Gbps host links (Figure 1). One "long" flow traverses all N bottlenecks; one "cross" flow traverses each single bottleneck. Flows are 50 MB by default and start simultaneously at $t = 0$. With two flows sharing each 25 Gbps link, the max-min fair rate is 12.5 Gbps for every flow. All experiments are deterministic: each configuration produces byte-identical artifacts across repeated runs (verified).

Metric. We compare each flow's actual flow completion time (FCT) against an idealized **fluid max-min oracle** — a continuous bandwidth-sharing model with no protocol dynamics (no slow start, no queueing, no convergence delay) in which the bandwidth is allocated max-min fairly at every instant. We compute the allocation by *progressive filling* [5]: all active flows raise their rate in lockstep until some link saturates; the flows crossing that link are frozen at their current rate; and the process repeats on the remaining flows until every flow is frozen. This is the textbook algorithm for max-min fair rates. Because flows complete and depart over time, we run it *event-driven* — recomputing the allocation at each flow completion — so the oracle FCT correctly accounts for the bandwidth freed as short flows finish. The oracle is therefore the best FCT any rate-allocation scheme could achieve under max-min fairness, and is exact for our fluid topologies. The headline metric is the **unfairness ratio** $\max_{\text{path}} FCT/\text{mean}(\text{short-path FCT})$; the oracle's value is 1.0 for equal-size, simultaneous flows.

[1] https://github.com/UNOCourseDemo/hpcc-fs

2.3 Findings F1–F4: characterizing the gap

Magnitude. Stock HPCC's unfairness ratio at $N \in \{2, 3, 4\}$ is 1.90, 1.92, 1.96: the multi-bottleneck flow's FCT is roughly twice that of single-bottleneck competitors. **There is a non-monotonic regime change at** $N \geq 5$, where the long flow becomes favored (0.51×); this turns out to be an INT-overflow artifact at `maxHop = 5` and we exclude $N \geq 5$ from the rest of this paper's stock comparisons (Figure 2).

Robustness. Perturbations across flow size and start time give penalties of 1.81×–3.53× (Figure 4): stable at 1.8–2.0× and *worsening* dramatically (3.5×) for small multi-bottleneck flows — the latency-sensitive RPC / control case. Even an established long flow gets displaced when cross flows arrive.

HPCC's own multi-rate mode. Multi-rate HPCC (per-hop EWMA, sender takes $\min_i Rc_i$) helps but does not fix the gap: 1.44/1.50/1.75× at $N = 2/3/4$ — still growing with bottleneck count.

RTT-equalization rules out RTT unfairness. Sweeping the bottleneck link delay while holding N fixed, the penalty is essentially flat at ∼1.9× across cross-flow RTTs spanning 10× the long flow's RTT (including cases where cross flows have *longer* RTT than the long flow). The multi-bottleneck penalty is **not** RTT unfairness.

Trace diagnosis. A per-flow throughput trace at $N = 4$ (Figure 5 left) shows the qualitative failure: the long flow collapses to ∼0.38 Gbps while the four cross flows hold ∼23.2 Gbps each for the entire 18 ms contention window; only after the cross flows complete does the long flow take the pipe. There are zero PFC events and zero loss. The collapse is "winner-take-all": at every shared link the single-bottleneck cross flow wins locally and the long flow is squeezed.

3 Why Sender-Only Fixes Are Insufficient

A natural first design instinct — and the one our project explored exhaustively — is to keep all state and computation at the sender, since HPCC's value proposition is its stateless switch design. We tried five sender-only correction variants behind a new `mb_mode` config knob, on top of unchanged HPCC. Each is a one- or two-line change in `UpdateRateHp`; the `mb_mode = 0` path is byte-identical to stock HPCC (verified).

3.1 The five variants

(1) **k-aware additive increase** (`mb_mode 1`). The flow counts congested hops k from INT and scales R_{AI} by $\gamma \cdot k$. Result at $N = 4$: penalty drops from 1.96 to 1.63× at $\gamma = 50$, still nowhere near 1.0.

(2) **Debiased max/mean blend** (`mb_mode 2`). $U_{\text{eff}} = (1-\lambda) \max u_i + \lambda \text{mean}_{u_i \geq \eta} u_i$. Result: penalty barely moves (1.95× at $\lambda = 1$),

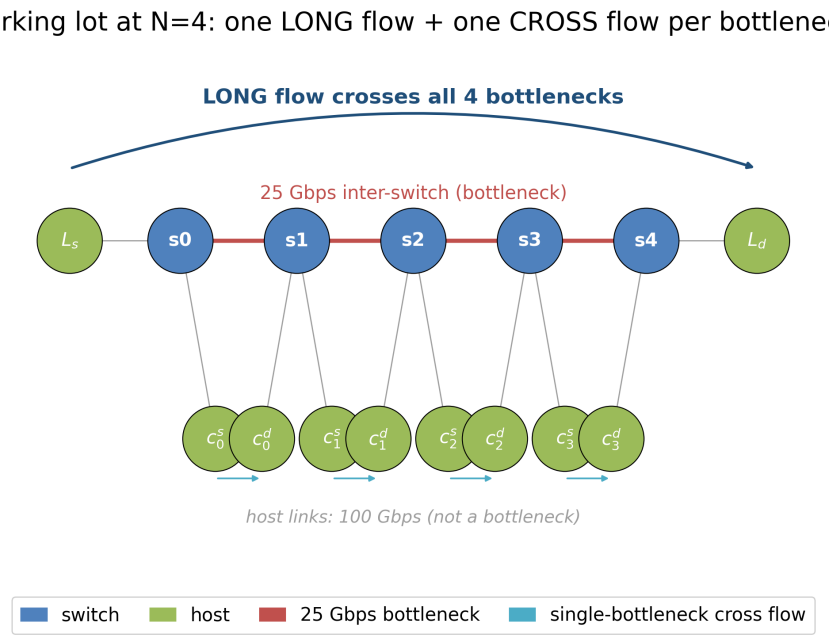


Figure 1: Parking-lot benchmark at $N = 4$. The long flow $L_s \to L_d$ crosses every bottleneck; each cross flow $c_i^s \to c_i^d$ traverses exactly one. Host links are not bottlenecks.

Table 1: Sender-side ablation: best penalty at $N = 4$ for each variant.

mb_mode	mechanism	best penalty
0	stock HPCC	1.96×
1	k-aware AI ($\gamma = 50$)	1.63×
2	debiased blend ($\lambda = 1$)	1.95×
3	responsibility-weighted MD	1.89×
4	k-th-root MD	1.92×
5	global share-aware AI ($\gamma = 20$)	1.71×

revealing that per-hop u_i are nearly equal in our setting — there is no significant max-bias to debias.

(3) **Responsibility-weighted MD** (mb_mode 3). The flow's multiplicative decrease is scaled by its measured share $own\_rate/txRate_{\text{btl}}$ at the bottleneck. Result: 1.89× — almost no effect, because the MD branch barely fires at the η-hold equilibrium.

(4) **k-th-root MD** (mb_mode 4). $new\_rate = curRate \cdot (1/\max c)^{1/k}$. Result: 1.92×.

(5) **Global share-aware additive increase** (mb_mode 5). Applied to *every* flow: $R_{AI}^{\text{eff}} = 2(1 - share)\gamma R_{AI}$ when congested. Result: 1.71× at $\gamma = 20$.

3.2 The structural reason

Two observations explain the pattern.

The η-hold equilibrium. Once the contended links settle at the target utilization η, HPCC's multiplicative term $1/\max c \approx 1$, so the rate update is dominated by holding. Every flow holds its current rate; if the dominant flows held a high rate at startup, they keep it. Additive nudges (modes 1, 5) take many tens of thousands of RTTs to redistribute meaningfully because R_{AI} is small compared to line rate. MD-side modifications (modes 3, 4) almost never fire because the multiplicative-decrease branch rarely activates at the hold point.

The information asymmetry. Strong redistribution requires moving the rate *multiplicatively* toward each flow's max-min fair share C/N. A sender knows C (from INT) and its own rate, but not N — the link's flow count. The only N-free sender mechanism is AIMD-style additive nudging, which we have just shown is too slow.

This is precisely why RCP [9] and XCP [12] put the fair-share computation in the switch. Our results are consistent with the same conclusion for the HPCC family: across the five representative sender-side designs we evaluated, none converges to the max-min allocation under the η-hold dynamics. We present this as empirical evidence, not a formal impossibility result — a tight lower bound on what sender-only INT can achieve is left to future work.

4 HPCC-FS Design

The empirical and structural arguments of §3 imply that the fix must (a) make the *dominant* flows yield, not just nudge the victims, and (b) place a fair-rate *signal* at the switch, since the sender cannot recover the fair share from a load signal alone. We do not have the switch count flows and divide; instead it runs a standard RCP

control law on aggregate load and queue (below) whose fixed point *is* the max-min fair rate C/N — the switch converges to the fair share without ever materializing N. We take the most minimal mechanism consistent with these constraints.

Background: RCP. The Rate Control Protocol (RCP) [9] is a router-assisted congestion control in which each link advertises a *single* rate R to *all* flows crossing it, holding no per-flow state. The key point is that R is not an aggregate budget for the link; it is the rate *each individual flow* should send at, such that when all of them do, the link is exactly full and shared equally. A router raises R when its link has spare capacity and lowers it when a queue forms; routers stamp R into passing packets, each flow sends at the minimum R along its path, and that rate is echoed to the source. A single flow adopting R therefore does not seize the link — it takes its share, because R has already been driven down by the load the other flows place on the same port. Concretely, with N flows each sending R the offered load is NR, and the controller drives $NR \to C$ (idle link $\Rightarrow$ R rises; standing queue $\Rightarrow$ R falls), so at equilibrium $R \approx C/N$ — the max-min (processor-sharing) fair share, tracked *without ever counting* N: as flows arrive or leave, R self-adjusts (a lone flow's R rises to C; a third arrival drops everyone to $C/3$). RCP thus supplies exactly the missing ingredient from §3: a fair-share signal computed where the flow count is implicitly observable (the link), not at the sender. HPCC-FS runs this control law once per egress port and carries its R in the INT header; a flow's path-minimum R yields network-wide max-min fairness, since a flow bottlenecked elsewhere contributes less load here and the loop hands the slack to the rest.

4.1 The mechanism

The entire protocol is Algorithms 1 and 2. The switch maintains one scalar fair rate per egress port and stamps the path-minimum into a single header field; the sender reads that field from the returning ACK and adopts it. No per-flow switch state, no per-hop record, no receiver changes.

At the switch (Alg. 1). Each egress port keeps a single scalar fair rate R_j, refreshed once per control interval d ($\approx$ one RTT) by the standard RCP rule $R \leftarrow R\,(1 + (\alpha(C - y) - \beta q/d)/C)$, where y is the load observed over the last interval and q the queue backlog. The term $\alpha(C - y)$ chases spare capacity (positive while the link is under-utilized, pushing R up) and $\beta q/d$ brakes when a standing queue forms (pulling R down); they balance at $y \approx C$, $q \approx 0$, which is the $R \approx C/N$ fixed point above. We use the RCP defaults $\alpha = 0.4$, $\beta = 0.226$ (untuned; the result is insensitive to them, §5.6). The port's entire state is three numbers — R_j, the last update time, and a byte-counter snapshot — with *no* per-flow data.

On the wire. The FS INT mode (`IntHeader::FS`) carries one 64-bit field, `fairRate`; the serialized cost is 8 bytes per packet *independent of hop count* (vs. 8 bytes *per hop* for stock HPCC). Each switch min-aggregates R_j into this field (first hop initialises it), so the receiver sees the path-minimum fair rate and echoes the header back unchanged — no receiver changes.

At the sender (Alg. 2). The new ACK handler `HandleAckFs` reads `fairRate` from the returning ACK and sets the sending rate to it, clamped to $[R_{\min}, NICrate]$. That single assignment is the whole sender control loop.

Algorithm 1 HPCC-FS switch — per egress port j

State R_j (fair rate), t_j (last update), B_j (txByte snapshot)
Const C_j link capacity; α=0.4, β=0.226; d control interval ($\approx$ RTT); $R_{\min}$
procedure OnDequeue(pkt, j) ▷ every packet leaving port j
  if R_j uninitialised **then**
    $R_j \leftarrow C_j/2$; $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$ ▷ conservative start
  end if
  if $now - t_j \geq d$ **then** ▷ one control interval elapsed
    $y \leftarrow (txBytes_j - B_j) \cdot 8 \,/\, (now - t_j)$ ▷ observed load
    $q \leftarrow queueBytes_j \cdot 8$ ▷ backlog (bits)
    $R_j \leftarrow R_j \cdot \big(1 + (\alpha(C_j - y) - \beta\, q/d)/C_j\big)$ ▷ RCP update
    $R_j \leftarrow \text{clamp}(R_j, R_{\min}, C_j)$
    $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$
  end if
  if $pkt.fairRate = 0$ **then** ▷ first hop on the path
    $pkt.fairRate \leftarrow R_j$
  else
    $pkt.fairRate \leftarrow \min(pkt.fairRate, R_j)$ ▷ path-min
  end if
  $txBytes_j \leftarrow txBytes_j + \text{sizeof}(pkt)$
end procedure

Algorithm 2 HPCC-FS sender (receiver echoes the header unchanged)

on QP-create: $rate \leftarrow R_{\min}$ ▷ conservative start

procedure OnAck(a)
  if $a.fairRate = 0$ **then return** ▷ no signal yet
  end if
  $rate \leftarrow \text{clamp}(a.fairRate, R_{\min}, NICrate)$ ▷ adopt path-min fair share
end procedure

4.2 Startup smoothing

Two cold-start details matter for clean operation. **Switch:** initialize $R = C/2$ (the common-case two-flow fair share), not C — otherwise the first stamped rate is line rate. **Sender:** initialize `qp->m_rate` to `m_minRate` for `cc_mode` 11, not the NIC line rate, so the first RTT before any ACK has returned does not blast at line rate. A flow obtains a path-complete fair-rate signal in a *single* RTT — unlike stock HPCC, whose per-link INT estimate needs multiple rounds to reflect a multi-bottleneck path — but full convergence to the max-min allocation takes longer, as the per-port RCP estimate settles over several control intervals (Section 5: ~4 ms, i.e. $\sim 10^2$ RTTs at our 21 μs base RTT). HPCC-FS's advantage is therefore *fast, path-length-independent* convergence, not literal one-RTT convergence.

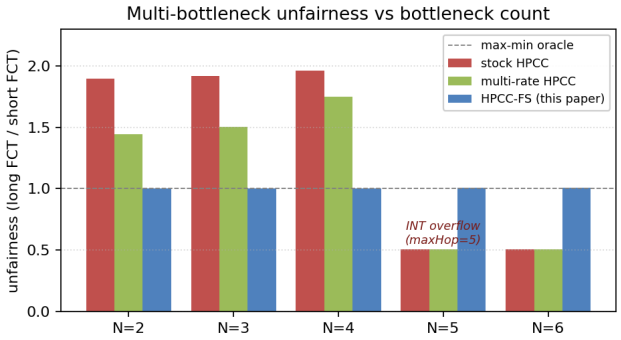


Figure 2: Multi-bottleneck unfairness vs bottleneck count N. HPCC-FS is flat ≈ 1.0 across the entire sweep; stock and multi-rate HPCC fail at $N \leq 4$ and break at $N \geq 5$ (INT-overflow regime).

4.3 Additive integration

HPCC-FS is `cc_mode == 11`. The mode-3 (HPCC) and mode-10 (PINT) wire formats and switch/sender code paths are completely untouched; we add a fourth case to (a) `IntHeader::Mode`, (b) its serialization, (c) the switch's mode-dispatch, (d) the sender's mode-dispatch, and (e) one branch in the validation driver. The driver also forces the QP window to 0 in FS mode (rate-only control), since `var_win` sizes the window from the NIC line rate rather than the bottleneck path BDP and would otherwise cap the multi-hop flow's effective window well below what is needed at its fair rate.

5 Evaluation

5.1 Methodology

All experiments use the deterministic parking-lot benchmark of §2, driven by our `hpcc-validation.cc`. Flows are 50 MB unless noted; sim time is 200 ms; the optimized binary is used throughout. We compare against stock HPCC (`cc_mode 3`, `multi_rate = false`), stock HPCC multi-rate, HPCC-PINT, and the `mb_mode` ablation suite. The simulator is a refactored re-port of the original HPCC ns-3 stack onto a tagged ns-3.45 release; before using it here we verified that the re-port still reproduces the original paper's baseline comparisons (HPCC vs. DCQCN, TIMELY, DCTCP, and PINT across WebSearch and FB-Hadoop workloads at 30–70% load on a 320-server fat-tree). The full reproduction record — per-run configurations, results, and analysis — ships with the artifact.

5.2 Headline: N-sweep

Stock and multi-rate HPCC are starved across $N = 2$–4 (penalty 1.90–1.96× for stock, 1.44–1.75× for multi-rate), then break entirely at $N \geq 5$, where the per-hop INT record overflows the `maxHop` cap (the apparent 0.51× there is the overflow artifact, not a real allocation). HPCC-FS is flat at 1.003–1.007× across the entire sweep — hop-count-independent by construction, since its single fair-rate field does not grow with path length. HPCC-PINT, which carries one compressed utilization field, inherits HPCC's load-based control and so still suffers the same ~1.8× (1.77–1.88×) multi-bottleneck penalty — confirming that the fix requires a fair-share signal, not merely a smaller INT footprint. All HPCC-FS runs have zero PFC events.

Table 2: Window ablation: HPCC-FS rate-only (default) vs. with the per-flow window cap re-enabled.

N	FS rate-only (default)	FS window-on
2	**1.003**×	1.50×
3	**1.003**×	2.00×
4	**1.005**×	2.49×

5.3 Structurally different topologies

Tree fabric (3-tier, no ECMP).. A 3-tier tree with unique shortest paths (15 nodes; 25 Gbps inter-switch, 100 Gbps host; Figure 3a). Stock penalty 1.36×, HPCC-FS **1.003**×, PFC = 0.

$k = 4$ fat-tree (with ECMP).. The canonical fat-tree (20 switches + 16 hosts; 48 links; Figure 3b) with the simulator's hash-based ECMP routing. Workload: 4 inter-pod flows (5-switch paths) plus 4 intra-pod different-edge flows (3-switch paths). We report two distinct quantities. (i) The *oracle-normalized penalty* — mean long-path slowdown divided by mean short-path slowdown, each measured against the max-min oracle — which removes the effect of ECMP load placement: stock 1.34×, HPCC-FS **1.001**× (every contended flow lands at its oracle FCT). (ii) The *raw* long/short FCT ratio, which under HPCC-FS is still ~1.32× — but this residual is *not* CC unfairness: ECMP hashes some flows onto under-loaded cores so they finish early, inflating the raw ratio independently of the congestion controller. PFC = 0 for both schemes. The point is that, once ECMP load placement is normalized away, HPCC-FS removes the multi-bottleneck penalty; the path-min fair-rate aggregation is path-invariant, so ECMP requires no design changes.

5.4 Robustness across asymmetric workloads

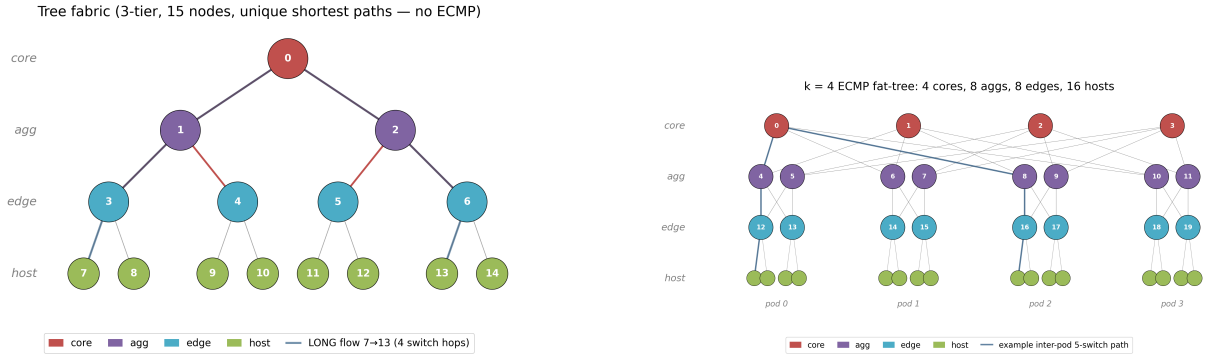
Across six perturbations (Figure 4), HPCC-FS holds within 1.0 ± 0.012. The worst stock case (small multi-bottleneck flow, 3.53× penalty) drops to 1.012×.

5.5 Ablation: rate-only is necessary

HPCC-FS runs rate-only in FS mode — the per-flow window cap is disabled (§4). Table 2 ablates this: re-enabling the window cap in FS mode degrades the penalty to 1.5–2.5×, *worsening with path length* (at $N = 4$ it is worse than stock HPCC). The cause is that `var_win` sizes the window from the NIC line rate, not the smaller multi-hop bottleneck path, so the long flow is window-bound below its fair rate. Disabling the cap is thus a necessary part of the design, not an incidental change; the switch's RCP rate already keeps queues bounded (zero PFC, §5.7).

5.6 Parameter sensitivity

A reasonable concern is that HPCC-FS's RCP gains are tuned to the benchmark. They are not: we use the standard RCP defaults ($\alpha = 0.4$, $\beta = 0.226$) untouched. Sweeping each parameter *one at a time* at $N = 4$ on the equal-size parking-lot (others at default), the penalty stays within $[1.004, 1.008]\times$ with zero PFC throughout:



(a) Tree fabric — unique shortest paths, no ECMP.

(b) $k = 4$ ECMP fat-tree (4 pods).

Figure 3: Two structurally different topologies used to test generalization beyond the parking lot. (a) 3-tier tree fabric, 15 nodes; the highlighted line is the long flow $7 \rightarrow 13$, crossing four switches. (b) Canonical $k = 4$ fat-tree (4 cores, 8 aggs, 8 edges, 16 hosts in 4 pods); the highlighted line is a representative inter-pod 5-switch path.

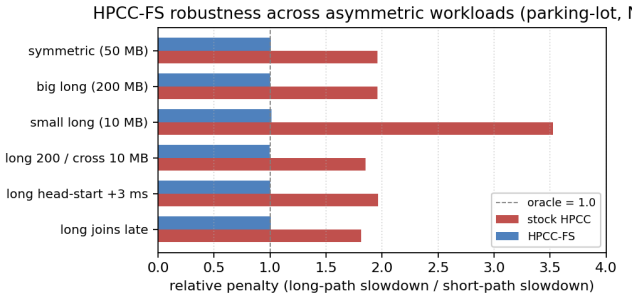


Figure 4: Robustness across asymmetric-size and staggered-start variants at $N = 4$. HPCC-FS is within 1.0 ± 0.012 across all six scenarios, including the small-long worst case where stock reaches 3.53×.

$\alpha \in [0.2, 0.6]$ (1.004–1.007×), $\beta \in [0.1, 0.4]$ (1.005×, flat), the startup fraction $R_0/C \in [0.25, 1.0]$ (1.004–1.008×; even $R_0 = C$ is PFC-free, because the sender's `m_minRate` start absorbs the first-interval burst, §4), and *minRate* $\in [100, 2000]$ Mb/s (1.005×, flat). Within this benchmark family the RCP parameters are not load-bearing; we do not claim the same insensitivity for arbitrary topologies, joint sweeps, or heterogeneous link speeds, which remain future work.

5.7 Queueing and PFC

With the smoothed startup (§4.2), HPCC-FS produces **zero PFC events** on every workload in the N-sweep and the robustness matrix. ECN marking remains; queue length is bounded (peak ≈120 KB at $N = 4$); no flow loss.

5.8 Do-no-harm

The HPCC algorithm-validation smoke configuration (`cc_mode` 3, single-bottleneck incast) passes all checks: 6/6 flows complete, 0 PFC, validation artifacts byte-identical to pre-modification. The stock `cc_mode` 3 N-sweep is bit-identical to the pre-change baseline.

Table 3: HPCC's home turf (single-bottleneck incast). HPCC-FS holds far less queue than HPCC at every startup, and a moderate startup also matches or beats HPCC's FCT. The same startup leaves the multi-bottleneck fairness headline unchanged (1.003–1.004×, PFC = 0).

	mean FCT	tail FCT	peak queue
HPCC	258.6 μs	459.8 μs	143 KB
HPCC-FS (startup 1 Gbps)	297.5 μs	490.3 μs	**64 KB**
HPCC-FS (startup 5 Gbps)	271.2 μs	464.3 μs	**49 KB**
HPCC-FS (startup 8 Gbps)	**242.0 μs**	**416.8 μs**	**38 KB**

The HPCC-FS additions affect the `cc_mode` 3 path only by adding new code; no existing function's output for existing inputs is altered.

5.9 HPCC's home turf: competitive FCT, lower queue

On contended flows HPCC-FS uses RCP-style switch-computed rates rather than HPCC's per-hop control, so a fair question is what this costs on *single*-bottleneck workloads — HPCC's home turf. We measure the validated incast benchmark (3 senders → 1 receiver over a 25 Gbps bottleneck; 12 flows, 100–400 KB), sweeping the sender's startup rate (short flows begin at the startup rate and ramp until the first fair-rate ACK returns):

Two things stand out (Table 3). First, HPCC-FS holds *far less queue* than HPCC at *every* startup (38–64 KB vs 143 KB) — RCP's $\beta q/d$ term drains queue aggressively. Second, the only place HPCC leads is FCT at the conservative 1 Gbps startup (~15%); raising the startup closes and then *reverses* that gap — at 8 Gbps HPCC-FS beats HPCC's mean FCT (242 vs 259 μs) at roughly one-quarter the queue, with zero PFC. The startup is a clean knob (queue rises again only past the point where the aggregate startup rate exceeds link capacity), and crucially it does *not* disturb the multi-bottleneck headline: at 8 Gbps the parking-lot penalty stays 1.003–1.004× ($N = 2, 3, 4$) with robustness variants 1.002–1.009×, all PFC = 0 (the same operating point as the headline runs). So on the home-turf

workloads we measured, HPCC-FS's fairness fix comes at *little-to-no FCT cost and a queue benefit* — not a trade-off.

We are careful not to overclaim: HPCC-FS still lacks HPCC's per-hop, window-bounded reaction, so on rapidly-varying or adversarial-microburst workloads (which we did not evaluate) HPCC's precision may matter; recovering that while keeping the fairness fix is what the hybrid overlay of §7 targets.

5.10 Per-flow rate trace, before vs after

The before-vs-after rate trace (Figure 5) is the cleanest qualitative result. Stock HPCC: the long flow collapses to ~0.4 Gbps while the four cross flows hold ~23 Gbps each for the full 18 ms contention; HPCC-FS: all five flows converge to ~12.5 Gbps within ~4 ms and stay locked at fair share.

6 Related Work

Datacenter transport. A long line of datacenter congestion control trades buffering for latency. DCTCP [2] reacts to ECN-marked standing queue; DCQCN [27] and TIMELY [17] bring rate-based control to lossless RDMA fabrics using ECN and delay respectively; and Swift [14] shows a carefully engineered delay signal scales to large fabrics. Receiver- and scheduling-driven designs such as Homa [18] and pFabric [3] instead prioritize short flows for low FCT. HPCC [15] marked a shift toward *in-network telemetry*: rather than infer congestion from a single scalar, the sender reads exact per-hop link load and sizes its window precisely, and On-Ramp [16] and PowerTCP [1] push the precise-signal philosophy further. Across this line the optimization target is overwhelmingly single-bottleneck FCT, tail latency, or queueing; *fairness across multiple shared bottlenecks*—our focus—receives comparatively little attention, and we show it is exactly where HPCC's otherwise-precise control breaks.

In-network telemetry. INT [13] lets the data plane stamp per-hop state into packets on programmable pipelines, and HPCC was among the first to drive congestion control directly from it. The cost is header overhead, which later work minimizes: HPCC-PINT [25] compresses the per-hop record into a single probabilistic field, and Poseidon [24] restructures INT-based control for deployability on commodity hardware. HPCC-FS continues this minimal-telemetry direction but changes *what* the field carries: not a raw load sample to be interpreted by the sender, but an explicit fair *rate* computed at the switch and min-aggregated along the path—one 64-bit field, like PINT, yet a decision rather than a measurement.

Explicit-rate and switch-assisted control. Having the switch compute and return a sending rate is a classic idea. ATM ABR explicit-rate schemes such as ERICA [11] computed a fair share per port decades ago; XCP [12] decomposed switch feedback into separate efficiency and fairness controllers; and RCP [9] reduced this to a single fair-rate scalar per link with no per-flow state, packet-stamped, with senders taking the path minimum. RCP is the closest mechanism to ours: HPCC-FS adopts its update rule but situates it as a *minimal fairness signal injected into an existing INT load-signaling system* rather than a standalone protocol, keeping HPCC's single-rate sender and per-hop transport. ABC [10] argues, for wireless links, that switches should compute the right control signal rather than expose raw load; we apply the same philosophy with minimal departure from HPCC's datacenter design.

In-switch fairness. A parallel line enforces fairness inside the switch via scheduling or dropping. Fair queueing [8] and Deficit Round Robin [21] give each flow its own queue, while Core-Stateless Fair Queueing [23] and Approximate Fair Dropping [19] approximate fair bandwidth shares *without* per-flow queues—using a single estimated fair rate and probabilistic dropping, the same stateless-per-flow spirit as RCP and HPCC-FS. Programmable schedulers such as PIFO [22] and AIFO [26] make rich scheduling policies realizable at line rate. HPCC-FS differs in mechanism: rather than enforcing fairness by per-packet queueing or dropping, it *advertises* one fair rate per egress port and lets rate-controlled RDMA senders converge to it—one scalar of egress state, no per-flow tracking and no extra queues.

Programmable-switch feasibility. Reconfigurable match-action pipelines [7] and P4 [6] make the per-port computation HPCC-FS needs—an EWMA of egress load, a multiply–add rate update, and stamping one header field—implementable on the same class of programmable switches that HPCC's INT and Bolt [4] already assume.

Closely related switch-assisted DC schemes. Bolt [4] targets HPCC's latency limits with Sub-RTT Control, Proactive Ramp-Up, and Supply Matching on programmable switches, reporting ~3× FCT improvement and ~80% tail-latency reduction over HPCC at 400 Gbps; it focuses on tail latency and ramp-up rather than multi-bottleneck max-min fairness and keeps richer per-flow switch state than our single scalar per port. A direct head-to-head on the multi-bottleneck workload of §2 would clarify how Bolt's per-flow window adjustments interact with the dynamics we characterize. Annulus [20] uses a dual-control-loop design for traffic aggregates mixing WAN and datacenter flows—largely orthogonal to intra-DC multi-bottleneck fairness within a single class.

Fairness foundations. Max-min fairness and the progressive-filling characterization we use as our oracle are classical [5], and the parking-lot topology is a standard stress test for multi-bottleneck rate allocation. Our contribution is not a new fairness definition but the diagnosis that HPCC cannot realize this well-understood objective from sender-visible INT alone, and the demonstration that a single switch-computed fair rate restores it.

7 Discussion

Relation to HPCC, and a hybrid path. HPCC-FS is best understood not as a tweak to HPCC's control law but as a demonstration of *where* the multi-bottleneck fix must live: a fair-share signal computed at the switch, which we instantiate with RCP. On contended flows it therefore replaces HPCC's window-bounded, per-hop-INT control with an RCP-style rate, and inherits RCP's character — fair and low-queue, but without HPCC's aggressive single-flow precision (§5.9). The natural way to keep both is a *hybrid overlay*: run HPCC unchanged on every flow and use the switch's fair rate only as a per-flow *ceiling*, $rate = \min(rate_{\mathrm{HPCC}}, fairRate)$, so dominant flows are capped to their fair share (and yield bandwidth that

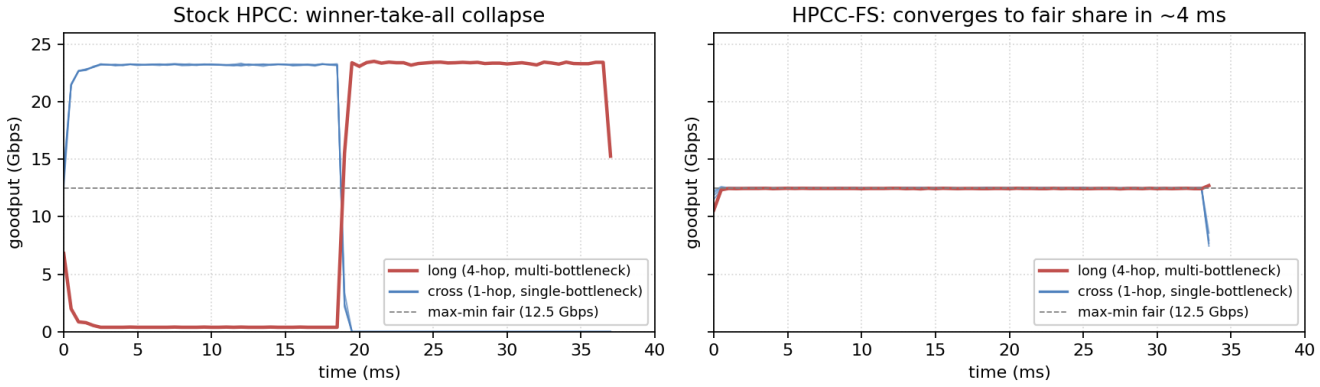


Figure 5: Per-flow goodput at $N = 4$, stock HPCC (left) vs HPCC-FS (right). Stock collapses the long flow to ~0.4 Gbps while cross flows hold ~23 Gbps for 18 ms (winner-take-all). HPCC-FS converges all five flows to the max-min fair share (12.5 Gbps) within ~4 ms.

HPCC's own increase gives to the starved flow) while HPCC's window and reaction are preserved. The open obstacle is the window–path mismatch behind our rate-only choice (§4): HPCC's window is sized for the NIC, not the multi-hop bottleneck path, so a victim flow may stay window-bound below fair even after the ceiling frees bandwidth — the overlay likely needs a path-aware window. We leave this hybrid, which would retain HPCC's precision *and* its fairness, to future work.

INT hop limit and the non-monotonic flip. The `maxHop` = 5 limit in the INT-header layout is the source of the non-monotonic regime change in stock HPCC: paths longer than five switches overflow the INT record and the rate-control input is corrupted. HPCC-FS sidesteps this because its wire format carries one field independent of hop count, and our $N = 5, 6$ measurements (§5) confirm it remains at penalty $\approx 1.0\times$ in that regime.

Mixed-size workloads. In a preliminary test on the parking-lot under a Web-Search size distribution (30 flows, 483 B to 9.6 MB), HPCC-FS reduces multi-bottleneck flow mean slowdown by ~15% but increases short-path mean slowdown by ~45%. This is correct max-min behaviour — stock HPCC's starvation of the long flow was inflating cross-link headroom that short flows benefited from — but it surfaces a workload-engineering question that max-min alone does not resolve. A defensible mixed-size evaluation needs per-class targets and is application-dependent; we leave it to follow-on work.

Scaling to larger fat-trees. The fat-tree generator runs at $k = 6$ (45 switches, 36 hosts) and $k = 8$ (80 switches, 64 hosts). At larger k the fabric has $(k/2)^2$ cores, so inter-pod flows spread across uncongested core paths and persistent contention does not materialize for our scaling workload — both stock and HPCC-FS show penalty <1, an ECMP load-distribution effect. HPCC-FS adds no overhead in this regime (0 PFC across all six runs). A workload designed to saturate larger fabrics with guaranteed inter-pod contention is open future work.

Heterogeneous link speeds. Our parking lot has uniform 25 Gbps bottlenecks. Mixed link speeds may interact with RCP's $\alpha(C - y)$ term in ways the symmetric case does not exercise. RCP itself is known to handle heterogeneity gracefully; we expect the same for HPCC-FS but have not verified.

Multi-path / ECMP.. §5 shows HPCC-FS works under ECMP on a $k = 4$ fat-tree with hash-based path selection — the path-min fair-rate aggregation is itself path-invariant. ECMP load imbalance does produce per-flow FCT spread, but this is a routing issue, not a CC issue.

Hardware feasibility. RCP's update is a single per-port floating-point or fixed-point step with three multiplies and an add; it is well within the budget of programmable switches. The min-aggregation per packet is a 64-bit compare and conditional write.

Limitations. Three topology families (parking-lot, tree-fabric, $k = 4$–8 fat-tree) and one in-sim baseline (HPCC-PINT); no head-to-head with hardware-assisted designs such as Bolt or PowerTCP, which would require re-implementing them in this simulator; no testbed measurements; convergence is a few ms (not one RTT, §4); and the ECMP evaluation is a small controlled workload, not a realistic mixed-size + ECMP combination. We did, however, verify that the result is insensitive to the RCP parameters (§5.6), so the absence of tuning is not a hidden confound.

8 Conclusion

We have shown that HPCC's load-signaling INT design has a structural fairness failure in multi-bottleneck workloads — a real, robust, RTT-independent winner-take-all collapse with ~2× FCT penalty for the multi-bottleneck flow, up to 3.5× for small such flows. No sender-side correction we attempted (across five representative variants) closes the gap, because at the η-hold equilibrium additive nudges are too slow and the sender cannot compute its fair share without knowing the link's flow count. A minimal extension that suffices on our benchmarks is **one fair-rate field** in a new INT mode plus **one scalar of state per switch port** (and, in our prototype, disabling the per-flow window cap in FS mode — see Section 4). The resulting protocol, HPCC-FS, achieves max-min

fairness with penalty 1.005× and zero PFC at $N = 4$ across all workloads we tested, converging in a few milliseconds independent of path length, while leaving the original HPCC code path unchanged.

References

[1] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. 2022. PowerTCP: Pushing the Performance Limits of Datacenter Networks. In *Proc. USENIX NSDI*. USENIX Association, Berkeley, CA, USA, 51–70.

[2] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data Center TCP (DCTCP). In *Proc. ACM SIGCOMM*. ACM, New York, NY, USA, 63–74.

[3] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pFabric: Minimal Near-Optimal Datacenter Transport. In *Proc. ACM SIGCOMM*.

[4] Serhat Arslan, Yuliang Li, Gautam Kumar, and Nandita Dukkipati. 2023. Bolt: Sub-RTT Congestion Control for Ultra-Low Latency. In *Proc. USENIX NSDI*. USENIX Association, Berkeley, CA, USA, 17 pages.

[5] Dimitri Bertsekas and Robert Gallager. 1992. *Data Networks* (2nd ed.). Prentice Hall, Englewood Cliffs, NJ, USA.

[6] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014).

[7] Pat Bosshart, Glen Gibb, Hun-Seok Kim, George Varghese, Nick McKeown, Martin Izzard, Fernando Mujica, and Mark Horowitz. 2013. Forwarding Metamorphosis: Fast Programmable Match-Action Processing in Hardware for SDN. In *Proc. ACM SIGCOMM*.

[8] Alan Demers, Srinivasan Keshav, and Scott Shenker. 1989. Analysis and Simulation of a Fair Queueing Algorithm. In *Proc. ACM SIGCOMM*.

[9] Nandita Dukkipati. 2008. *Rate Control Protocol (RCP): Congestion Control to Make Flows Complete Quickly*. Ph. D. Dissertation. Stanford University, Stanford, CA, USA.

[10] Prateesh Goyal, Akshay Narayan, Frank Cangialosi, Srinivas Narayana, Mohammad Alizadeh, and Hari Balakrishnan. 2020. ABC: A Simple Explicit Congestion Controller for Wireless Networks. In *Proc. USENIX NSDI*. USENIX Association, Berkeley, CA, USA, 353–372.

[11] Shivkumar Kalyanaraman, Raj Jain, Sonia Fahmy, Rohit Goyal, and Bobby Vandalore. 2000. The ERICA Switch Algorithm for ABR Traffic Management in ATM Networks. *IEEE/ACM Transactions on Networking* 8, 1 (2000).

[12] Dina Katabi, Mark Handley, and Charlie Rohrs. 2002. Internet Congestion Control for Future High Bandwidth-Delay Product Environments. In *Proc. ACM SIGCOMM*. ACM, New York, NY, USA, 89–102.

[13] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Antonin Bas, Advait Dixit, and Lawrence J. Wobker. 2015. In-band Network Telemetry via Programmable Dataplanes. In *Proc. ACM SIGCOMM (Industrial Demo)*.

[14] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *Proc. ACM SIGCOMM*.

[15] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. 2019. HPCC: High Precision Congestion Control. In *Proc. ACM SIGCOMM*. ACM, New York, NY, USA, 44–58.

[16] Shiyu Liu, Ahmad Ghalayini, Mohammad Alizadeh, Balaji Prabhakar, Mendel Rosenblum, and Anirudh Sivaraman. 2021. Breaking the Transience-Equilibrium Nexus: A New Approach to Datacenter Packet Transport. In *Proc. USENIX NSDI*.

[17] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. 2015. TIMELY: RTT-based Congestion Control for the Datacenter. In *Proc. ACM SIGCOMM*. ACM, New York, NY, USA, 537–550.

[18] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities. In *Proc. ACM SIGCOMM*.

[19] Rong Pan, Lee Breslau, Balaji Prabhakar, and Scott Shenker. 2003. Approximate Fairness through Differential Dropping. *ACM SIGCOMM Computer Communication Review* 33, 2 (2003).

[20] Ahmed Saeed, Varun Gupta, Bharath Balasubramanian Soman, Aditya Akella, Sumeet Rangwala, and Sangeetha Kannan. 2020. Annulus: A Dual Congestion Control Loop for Datacenter and WAN Traffic Aggregates. In *Proc. ACM SIGCOMM*. ACM, New York, NY, USA, 735–749.

[21] M. Shreedhar and George Varghese. 1995. Efficient Fair Queuing using Deficit Round Robin. In *Proc. ACM SIGCOMM*.

[22] Anirudh Sivaraman, Suvinay Subramanian, Mohammad Alizadeh, Sharad Chole, Shang-Tse Chuang, Anurag Agrawal, Hari Balakrishnan, Tom Edsall, Sachin Katti, and Nick McKeown. 2016. Programmable Packet Scheduling at Line Rate. In *Proc. ACM SIGCOMM*.

[23] Ion Stoica, Scott Shenker, and Hui Zhang. 1998. Core-Stateless Fair Queueing: Achieving Approximately Fair Bandwidth Allocations in High Speed Networks. In *Proc. ACM SIGCOMM*.

[24] Weitao Wang, Masoud Moshref, Yuliang Li, Gautam Kumar, T. S. Eugene Ng, Neal Cardwell, and Nandita Dukkipati. 2023. Poseidon: Efficient, Robust, and Practical Datacenter CC via Deployable INT. In *Proc. USENIX NSDI*.

[25] Liron Schiff Yang, Ran Ben Basat, Michael Mitzenmacher, Maria Apostolaki, Aviv Klein, Erez Saito, and Avi Mendelson. 2020. PINT: Probabilistic In-band Network Telemetry. In *Proc. ACM SIGCOMM*. ACM, New York, NY, USA, 662–677.

[26] Zhuolong Yu, Chuheng Hu, Jingfeng Wu, Xiao Sun, Vladimir Braverman, Mosharaf Chowdhury, Zhenhua Liu, and Xin Jin. 2021. Programmable Packet Scheduling with a Single Queue. In *Proc. ACM SIGCOMM*.

[27] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proc. ACM SIGCOMM*. ACM, New York, NY, USA, 523–536.